Helwan University
Faculty of Engineering — Computer & Systems Dept.

Assignment 01

To-Do List Mobile Application

Built with React Native & Expo

Submitted by

Eman Ahmed Ramadan Mahmoud

Course: Mobile Computing

Instructor: Eng. Rania El-Sayed

Academic Year 2025 – 2026

1. Introduction

This report documents the development of a To-Do List mobile application built using React Native with the Expo framework. The application fulfills the requirements of Assignment 01, which focuses on fundamental React Native concepts including state management, component usage, and list rendering.

2. Assignment Requirements

2.1 Logic Requirements

The assignment required building a simple To-Do list application that includes the following components:

- Text component — for displaying the app title
- TextInput component — for entering new task goals
- Button / TouchableOpacity — for adding items to the list
- FlatList component — for rendering the list of goals with scrolling support

Additionally, the app requires:

- Two state variables to handle: text input changes and adding items to the list
- One function to handle the button click event

2.2 Design Requirements

- Use a consistent color palette throughout the app
- Custom fonts integrated via the Expo Fonts API
- Feel free to improve beyond the classic design provided

2.3 Deliverables

- Report including: color palette, fonts, and screenshots of different app states
- Code uploaded to Expo Snack with link in GitHub repository
- 2-minute video showing the app in runtime

3. Implementation

3.1 Components Used

Component	Purpose
`Text`	Displays app title "My Tasks" and task content
`TextInput`	Input field for entering new task descriptions
`TouchableOpacity`	Used for the Add button, checkbox toggle, delete button, and filter tabs
`FlatList`	Renders the scrollable list of tasks with keyExtractor and renderItem
`KeyboardAvoidingView`	Prevents the keyboard from covering the input field on iOS/Android
`StatusBar`	Configures the system status bar style to match the dark header
`ActivityIndicator`	Shown while custom fonts are loading

3.2 State Variables

State Variable	Type	Purpose
`inputText`	String	Tracks the current text typed in the TextInput field
`tasks`	Array	Stores all task objects, each with id, text, and done status
`filter`	String	Controls which tasks are shown: "all", "active", or "done"

3.3 Handler Functions

- addTaskHandler() — Validates input, creates a new task object with a unique ID, and prepends it to the tasks array
- deleteTaskHandler(id) — Filters out the task with the matching ID from the tasks array
- toggleDoneHandler(id) — Toggles the done boolean on the matching task, marking it complete or active
- clearDoneHandler() — Removes all completed tasks from the list and resets the filter to "all"

4. Design

4.1 Color Palette

The application uses a professional Navy & Gold color scheme with consistent usage across all UI elements:

Swatch	Name	Hex Code	Usage
	Navy (Primary)	`#1E2A4A`	Header, active buttons, borders
	Navy Dark	`#141E35`	Status bar background
	Navy Light	`#EEF0F8`	Loading background, alternating rows
	Gold (Accent)	`#F5A623`	Checkboxes, add button, active stat, left task border
	Gold Light	`#FFF8EC`	Done task background
	Background	`#F5F6FA`	App background, input container
	White	`#FFFFFF`	Active task cards, filter buttons
	Text Dark	`#1E2A4A`	Task text, headings
	Text Mid	`#666666`	Done task text (strikethrough)
	Text Light	`#999999`	Placeholder text, disabled button
	Red (Danger)	`#FF3B30`	Delete button icon
	Red Light	`#FFF0F0`	Delete button background, Clear Done button background

4.2 Typography

The app uses the Poppins font family loaded via @expo-google-fonts/poppins, following Expo's custom fonts documentation.

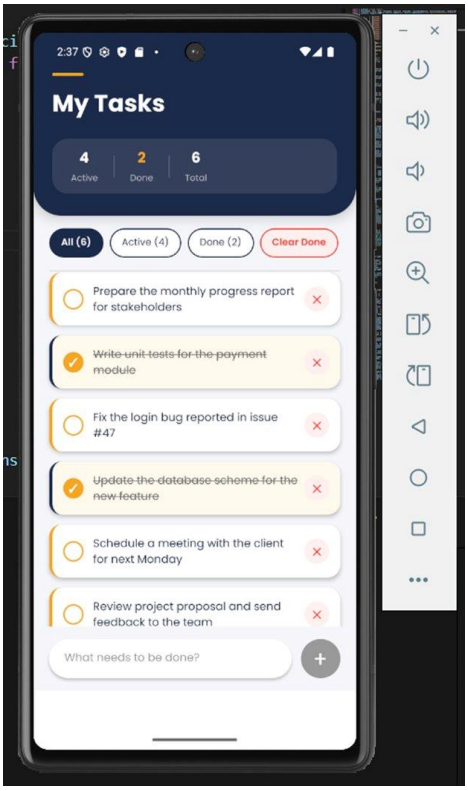
Font Variant	Weight	Used For
`Poppins_700Bold`	Bold (700)	App title, stat numbers
`Poppins_600SemiBold`	Semi-Bold (600)	Filter button active text, empty screen subtitle, clear done button
`Poppins_400Regular`	Regular (400)	Task text, filter labels, stat labels, placeholder text

5. App Screenshots

The following screenshots demonstrate the different states and interactions of the application.

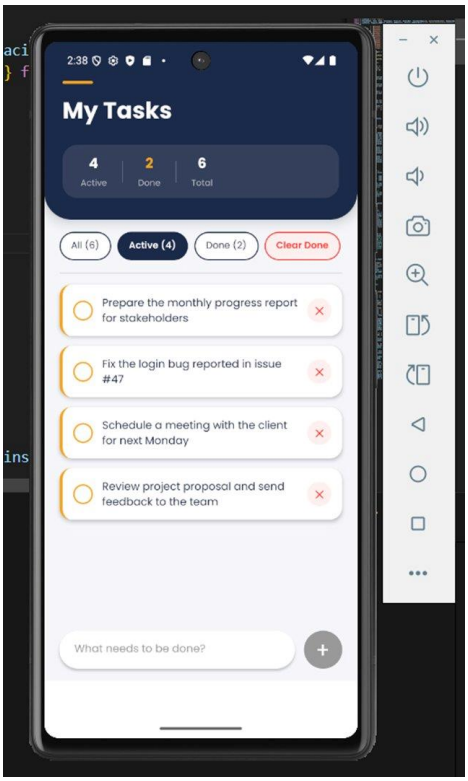
5.1 All Tasks View

Shows all 6 tasks (4 active, 2 completed) with the "All" filter selected. Completed tasks appear with a gold checkmark, strikethrough text, and a warm cream background. Active tasks have a gold left border and an empty circle checkbox.



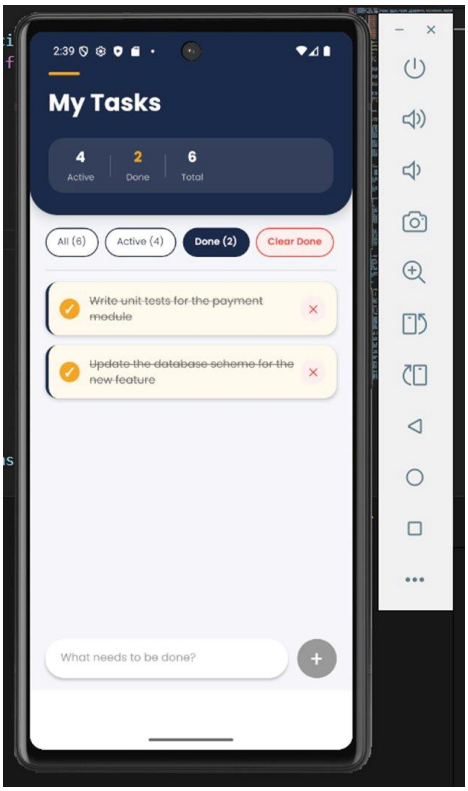
5.2 Active Filter View

Demonstrates the filter functionality — when the "Active" button is pressed, only the 4 incomplete tasks are displayed. The filter button turns navy with white text to indicate selection. The stats row in the header remains unchanged, always showing the total counts.



5.3 Done Filter View

Shows only the 2 completed tasks when the "Done" filter is selected. The "Clear Done" button (in red) allows the user to remove all completed tasks at once. Both completed tasks display with strikethrough text and the gold filled checkbox.



6. Extra Features Beyond Requirements

The implemented application goes beyond the basic assignment requirements with several additional features:

- Task Completion Toggle — Users can mark tasks as done/undone by tapping the circular checkbox, with visual feedback (strikethrough, background change, gold fill)
- Delete Individual Tasks — Each task has a red delete button (×) to remove it from the list instantly
- Filter Tabs — Three filter buttons (All, Active, Done) allow users to view tasks by status, with live counts updated in real time
- Clear Done Button — A dedicated red button appears dynamically when there are completed tasks, allowing batch removal
- Live Statistics Header — The header displays real-time counts for Active, Done, and Total tasks, always kept in sync with the list
- Custom Fonts — Poppins font family integrated via @expo-google-fonts for a polished, professional look
- Keyboard Avoiding View — The input area stays visible when the keyboard is open on both iOS and Android
- Disabled Add Button — The add button is visually grayed out and non-interactive when the input field is empty
- Empty State Screen — A friendly emoji and contextual message is shown when no tasks match the current filter
- Consistent Navy & Gold Design System — A defined color palette applied consistently throughout all components

7. Project Links

All project deliverables are available through the following links:

Resource	Link
GitHub Repo	https://github.com/emanahmed845/Eman-Ahmed-Ramadan-Mahmoud_To-Do-Task
Expo Snack	https://snack.expo.dev/@eman_ahmed_ramadan/eman-ahmed-ramadan_to-do-ass
Demo Video	https://drive.google.com/file/d/1mFuMa1IfGammHBGmAk9TR_mPWNaH92Kz/view?usp=sharing
Drive Folder	https://drive.google.com/drive/folders/1iiqMu-CFXIKfzg6igM5tuSpB215YKEec?usp=sharing

8. Conclusion

The To-Do List application successfully fulfills all the requirements of Assignment 01. It demonstrates a solid understanding of core React Native concepts including state management with useState, component composition, FlatList rendering, and event handling. The app was designed with a consistent and modern color scheme, integrated custom fonts via Expo, and implemented several additional features to improve usability and user experience beyond the basic specification.